University of Moratuwa

EN3150 - Pattern Recognition

Assignment 03

Resource-Constrained CNN for Edge Image Classification

Group: Draco

Dealing with AI from the core

GUNARATHNA K.T.M.B.	230203G
JAYAWARDANA K.P.P.	230294H
JITHMAL P.G.I.	230306B
NIMADITH W.N.A.C.	230442L

Repository:

`https://github.com/NavigatorPrime/CNN-Image-Classifier`

October 2026

Contents

1 Introduction **1**

2 Task 1: Data Preparation **1**

2.1 Corrected common split 1

3 Task 2: Custom Architecture Design **3**

3.1 Model A: standard CNN 3

3.2 Model B: depthwise-separable CNN 3

4 Task 3: Optimizer Selection and Tuning **5**

4.1 Controlled comparison 5

4.2 Momentum effect 5

4.3 Why Adam and learning rate 0.001 were selected 5

5 Task 4: Custom Model Training and Evaluation **6**

5.1 Model A versus Model B trade-off 8

6 Task 5: Lightweight Pre-trained Models **8**

7 Task 6: Final Comparison and Trade-off Discussion **11**

7.1 Accuracy, storage, and computation 11

7.2 Accuracy-efficiency frontier 13

7.3 Engineering recommendations 13

8 Conclusion **13**

1 Introduction

Running image classifiers at the edge requires a balance between recognition accuracy and the memory and computation available on the target device. In this assignment, we investigate that balance using the five-class DeFungi dataset. We first develop two custom convolutional neural networks (CNNs) under the specified parameter budget, then compare them with fine-tuned MobileNetV2 and ShuffleNetV2 1.0x. To make the results comparable, all final evaluations use the corrected, leakage-free dataset split and the clean-split Stage 2 experiments.

The reported measurements were checked against saved JSON and CSV summaries, classification reports, training histories, checkpoints, and profiler outputs using `scripts/generate_report_data.py`. All seven experiments record the same package SHA-256 and manifest SHA-256 in their JSON metadata, and their training histories each cover 20 epochs. Each final confusion matrix contains 1,366 test images. This provides a consistent basis for the model comparisons without using any values from the earlier contaminated split.

Computational measurement. For an input of $1 \times 3 \times 64 \times 64$, we count multiply-accumulate operations (MACs) in Conv2d and Linear layers. BatchNorm, activations, pooling, residual additions, concatenation, channel shuffle, softmax, and data movement are excluded. FLOPs are calculated as $2\times$ MACs throughout the report. These figures describe arithmetic workload, not measured inference time on a physical device.

2 Task 1: Data Preparation

2.1 Corrected common split

The UCI DeFungi dataset contains five classes: H1, H2, H3, H5, and H6. All images in the prepared package are RGB and have a resolution of exactly 64×64 pixels. Before dividing the data, we checked for duplicate content in two ways: SHA-256 hashes of the original files and SHA-256 hashes of the decoded pixel arrays. This identifies both byte-identical files and images with identical pixel content despite possible file-level differences. We then performed a deterministic, class-aware allocation, keeping every member of each identical-content group in the same split. This prevents the model from being evaluated on a duplicate of an image it saw during training.

Table 1: Corrected leakage-free dataset split.

Split	H1	H2	H3	H5	H6	Total
Train	3,083	1,634	573	573	517	6,380
Validation	661	350	123	123	111	1,368
Test	660	350	123	122	111	1,366

The resulting training, validation, and test proportions are 70.00%, 15.01%, and 14.99%, respectively. We identified 59 duplicate-content groups within the dataset, but none crosses between the three partitions: there are zero cross-split duplicate groups under both the raw-byte and decoded-pixel SHA-256 checks. The dataset manifest also matches the packaged files, with 9,114 images in total. These checks are important because even a small amount of duplicate leakage could give an overly

optimistic estimate of generalization.

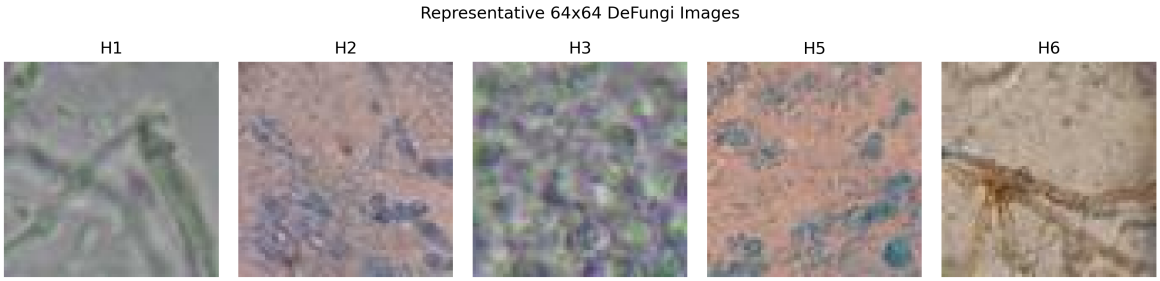


Figure 1: Representative 64×64 RGB images from the corrected training split.

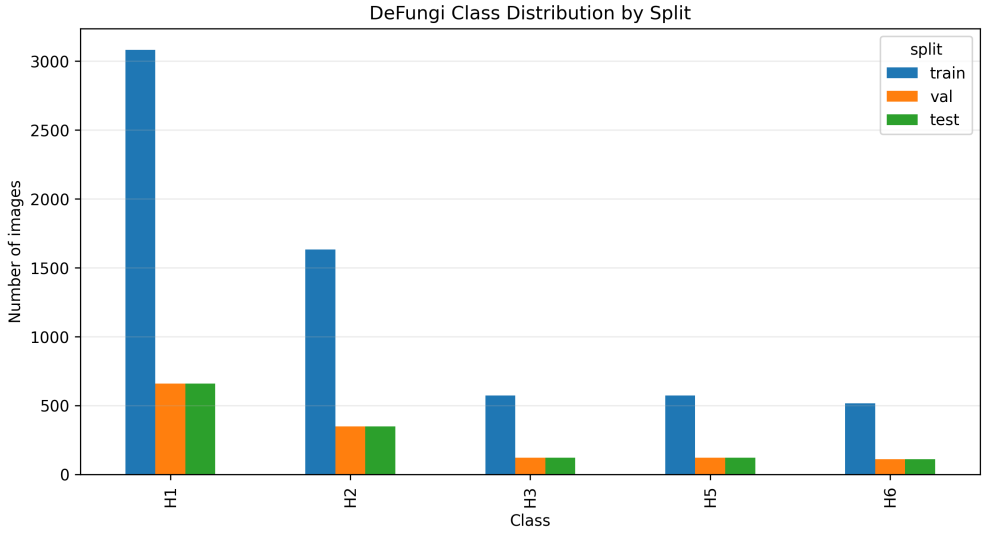


Figure 2: Per-class distribution in the corrected common split. H1 and H2 dominate, so macro-averaged metrics are reported alongside accuracy.

For training, images are resized to 64×64, followed by random horizontal and vertical flips, rotations of $\pm 15°$, color jitter, tensor conversion, and ImageNet normalization. These augmentations introduce useful appearance variation while leaving the class labels unchanged. Validation and test images undergo deterministic preprocessing without augmentation, so differences in their scores reflect model behavior rather than random image transformations. We use seed 42 to make the experimental setup reproducible.

3 Task 2: Custom Architecture Design

3.1 Model A: standard CNN

Model A serves as the conventional CNN baseline. It has four feature-extraction blocks, each consisting of a standard 3×3 convolution, BatchNorm, ReLU, and 2×2 max pooling. As the spatial dimensions decrease, the channel count increases to capture richer features. The implemented layers and their parameter counts are detailed in Table 2.

Table 2: Model A architecture. ReLU, pooling, flattening, and dropout have no trainable parameters.

Stage	Layer sequence	Input	Output filters/units	Kernel	Stride/padding	Output shape	Conv/FC	BN
Block 1	Conv2D, BN, ReLU, MaxPool	$3 \times 64 \times 64$	32	3×3; pool 2×2	1/1; pool 2	$32 \times 32 \times 32$	896	64
Block 2	Conv2D, BN, ReLU, MaxPool	$32 \times 32 \times 32$	64	3×3; pool 2×2	1/1; pool 2	$64 \times 16 \times 16$	18,496	128
Block 3	Conv2D, BN, ReLU, MaxPool	$64 \times 16 \times 16$	128	3×3; pool 2×2	1/1; pool 2	$128 \times 8 \times 8$	73,856	256
Block 4	Conv2D, BN, ReLU, MaxPool	$128 \times 8 \times 8$	256	3×3; pool 2×2	1/1; pool 2	$256 \times 4 \times 4$	295,168	512
Flatten	Flatten	$256 \times 4 \times 4$	4096	—	—	4096	0	0
FC1	Linear, ReLU, Dropout(0.5)	4096	512	—	—	512	2,097,664	0
FC2	Linear, ReLU, Dropout(0.3)	512	128	—	—	128	65,664	0
Output	Linear logits	128	5	—	—	5	645	0

For a biased Conv2D layer,

$$P_{\text{conv}} = (K_h K_w C_{\text{in}} C_{\text{out}}) + C_{\text{out}},$$

and trainable BatchNorm contributes $2C_{\text{out}}$ parameters (scale and shift). For example, Block 1 has $(3 \cdot 3 \cdot 3 \cdot 32) + 32 = 896$ convolution parameters and $2 \cdot 32 = 64$ BatchNorm parameters. The four blocks total 389,376 parameters. The classifier contributes

$$(4096 \cdot 512 + 512) + (512 \cdot 128 + 128) + (128 \cdot 5 + 5) = 2,163,973.$$

Therefore the verified total is $389,376 + 2,163,973 = \mathbf{2,553,349}$ trainable parameters.

After every convolutional BatchNorm layer and after the first two fully connected layers, Model A uses ReLU. Unlike sigmoid and tanh, ReLU only requires a simple threshold operation and avoids exponential evaluations. This is an attractive property for resource-limited hardware, although actual device latency was not measured in this study.

3.2 Model B: depthwise-separable CNN

To explore a more compact design, Model B replaces most standard convolutions with depthwise-separable operations. A standard convolutional stem is followed by six depthwise-separable blocks, global average pooling (GAP), dropout, and a linear classifier. Within each block, depthwise Conv2D, BatchNorm, and ReLU6 are followed by pointwise Conv2D, BatchNorm, and ReLU6. Separating spatial filtering from channel mixing reduces the parameter and arithmetic demands of the feature extractor.

Table 3: Implemented Model B architecture. Block parameter totals include both convolutions and both BatchNorm layers.

Stage	Input C	DW C	Groups	Kernel	Stride/pad	PW out	Output shape	Parameters	Activation
Stem	3	—	1	3×3	1/1	16	$16 \times 64 \times 64$	464	ReLU6
DS Block 1	16	16	16	3×3 DW, 1×1 PW	2/1	32	$32 \times 32 \times 32$	752	ReLU6 $\times 2$
DS Block 2	32	32	32	3×3 DW, 1×1 PW	1/1	32	$32 \times 32 \times 32$	1,440	ReLU6 $\times 2$
DS Block 3	32	32	32	3×3 DW, 1×1 PW	2/1	64	$64 \times 16 \times 16$	2,528	ReLU6 $\times 2$
DS Block 4	64	64	64	3×3 DW, 1×1 PW	1/1	64	$64 \times 16 \times 16$	4,928	ReLU6 $\times 2$
DS Block 5	64	64	64	3×3 DW, 1×1 PW	2/1	128	$128 \times 8 \times 8$	9,152	ReLU6 $\times 2$
DS Block 6	128	128	128	3×3 DW, 1×1 PW	1/1	128	$128 \times 8 \times 8$	18,048	ReLU6 $\times 2$
GAP	128	—	—	global	—	128	$128 \times 1 \times 1$	0	—
Dropout	128	—	—	—	—	128	128	0	—
Classifier	128	—	—	—	—	5	5 logits	645	Linear

The depthwise and pointwise convolution parameter formulas are

$$P_{\mathrm{DW}} = K_h K_w C_{\mathrm{in}} \quad \text{and} \quad P_{\mathrm{PW}} = C_{\mathrm{in}} C_{\mathrm{out}},$$

because both use `bias=False`. Their BatchNorm terms are $2C_{\mathrm{in}}$ and $2C_{\mathrm{out}}$. Thus a complete block has

$$P_{\mathrm{block}} = 9C_{\mathrm{in}} + 2C_{\mathrm{in}} + C_{\mathrm{in}} C_{\mathrm{out}} + 2C_{\mathrm{out}}.$$

For Block 1 this is $144 + 32 + 512 + 64 = 752$. For Block 2, whose output width is also 32, it is $288 + 64 + 1024 + 64 = 1,440$. The stem has $3 \cdot 16 \cdot 9 + 2 \cdot 16 = 464$ parameters and the classifier has $128 \cdot 5 + 5 = 645$. Summing every row gives:

$$\boxed{\text{Model B trainable parameters} = 37,957 < 100,000.}$$

The arithmetic benefit can also be seen analytically. Considering convolution alone, the ratio between depthwise-separable and standard convolution is $1/C_{\mathrm{out}} + 1/9$. For both the 16-to-32 and 32-to-32 blocks, this corresponds to 14.24%; the ratio becomes smaller for the 64- and 128-output blocks. As with all MAC calculations in this report, the comparison excludes BatchNorm and activation operations.

ReLU6 is applied after the stem and after both convolution stages in every separable block. It retains ReLU's simple piecewise-linear behavior while adding an upper clamp, without requiring exponential functions. At the output, a strictly linear layer produces five logits for `CrossEntropyLoss`; applying ReLU6 or softmax at that point would change the intended loss interface, so neither is included.

4 Task 3: Optimizer Selection and Tuning

4.1 Controlled comparison

We compared three optimizers on Model B to understand how the update rule affects convergence. For a fair comparison, all three trials shared the corrected dataset split, the same initial-state hash, seed 42, batch size 32, augmentation settings, cross-entropy loss, and weight decay 10^{-4}. Training lasted 20 epochs in each case, and the checkpoint with the highest validation accuracy was selected. We also applied `ReduceLROnPlateau` to validation loss with a reduction factor of 0.5 and patience 4; none of these runs reached a point where the scheduler reduced the learning rate.

Table 4: Optimizer results using the shared leakage-free split.

Optimizer	LR	Momentum	Best val. acc.	Epoch	Final train loss	Final val. loss	Test loss	Test acc.
Standard SGD	0.010	0.0	66.01%	18	0.8961	0.8384	0.8358	65.67%
SGD + Momentum	0.010	0.9	70.54%	18	0.7938	0.7165	0.7255	68.81%
Adam	0.001	–	**73.25%**	20	**0.7510**	**0.6671**	**0.6755**	**70.57%**

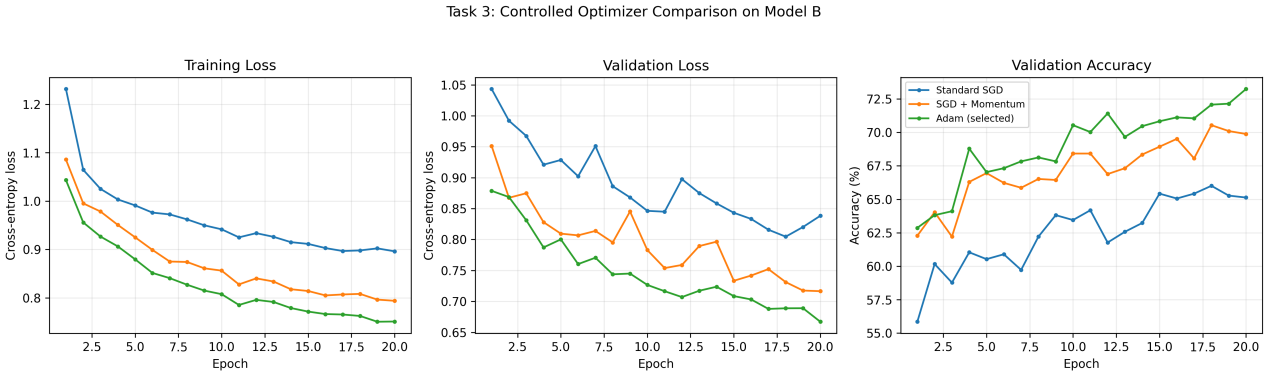


Figure 3: Training loss, validation loss, and validation accuracy over the controlled 20-epoch optimizer comparison. Each curve uses the same initial model state and corrected dataset split.

4.2 Momentum effect

Momentum accumulates a velocity in directions supported by successive gradients:

$$v_t = \mu v_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_t.$$

By carrying information from previous gradients into the next update, momentum can reduce oscillations and help the optimizer move more steadily in directions where the gradient is consistent. The measured curves reflect this advantage. At epoch 10, SGD with momentum reached 68.42% validation accuracy, compared with 63.45% for standard SGD. It also achieved a higher best validation accuracy (70.54% versus 66.01%), a lower final validation loss (0.7165 versus 0.8384), and better test accuracy (68.81% versus 65.67%).

4.3 Why Adam and learning rate 0.001 were selected

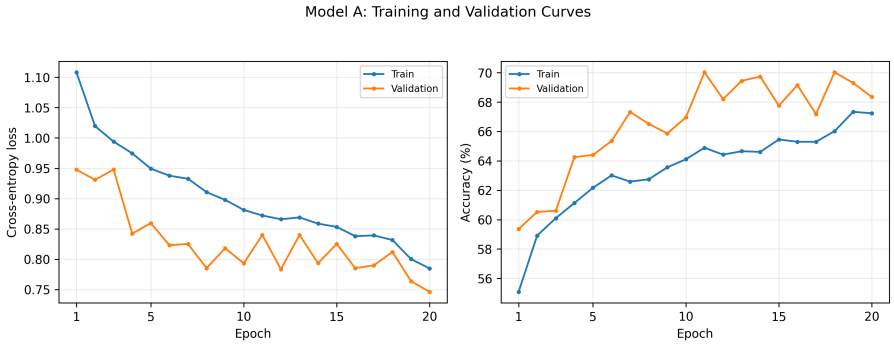
Adam gave the strongest results within the same 20-epoch training budget, with 73.25% best validation accuracy, 70.57% test accuracy, and macro F1 0.6800. These results were higher than those of either

SGD variant. The learning curves also show continued progress: validation accuracy reached 70.54% by epoch 10, and its best checkpoint occurred at epoch 20. Adam combines a first-moment estimate, similar in purpose to momentum, with second-moment scaling to adjust the effective update size for each parameter. This can make training less sensitive to differences in gradient magnitude across layers.

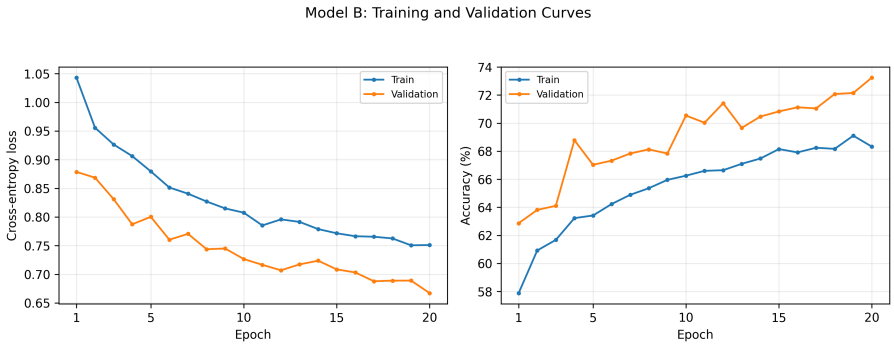
We therefore retained Adam with a base learning rate of 0.001, rather than the 0.01 used in both SGD trials. The smaller nominal rate works with Adam's adaptive scaling to support steady updates; in the actual run, it produced decreasing training and validation losses and the highest validation accuracy. Although a plateau scheduler was enabled, it did not lower the learning rate, so every recorded Adam epoch used 0.001. The optimizer choice is based on this controlled comparison, not on an assumption that Adam is always superior.

5 Task 4: Custom Model Training and Evaluation

After choosing the training configuration, we trained both custom models on the same corrected split for 20 epochs. For each model, we restored its selected checkpoint before evaluating the held-out test set. This avoids presenting the last epoch as the final model when an earlier checkpoint performed better on validation data.

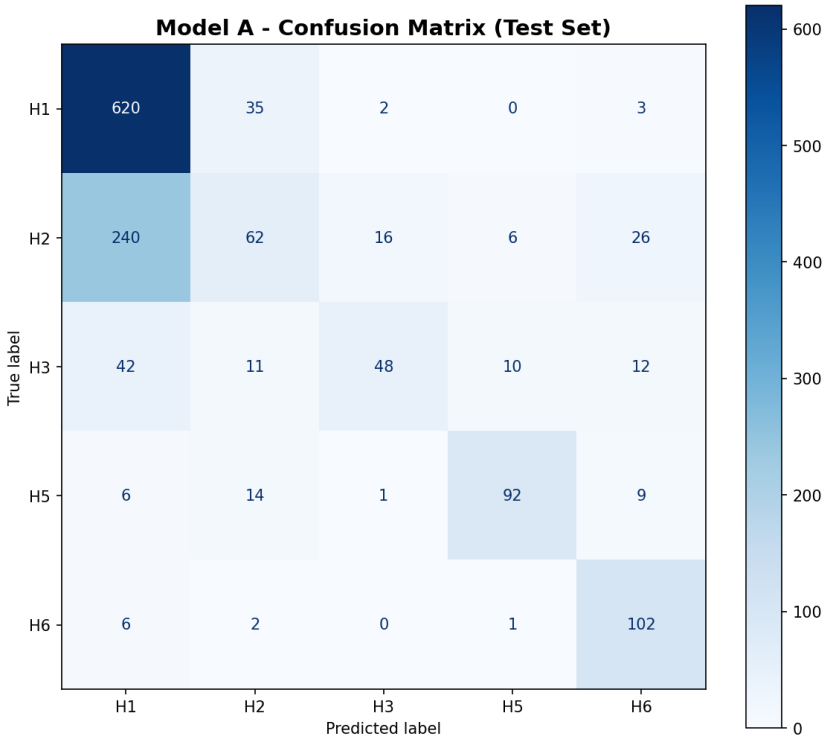


(a) Model A.

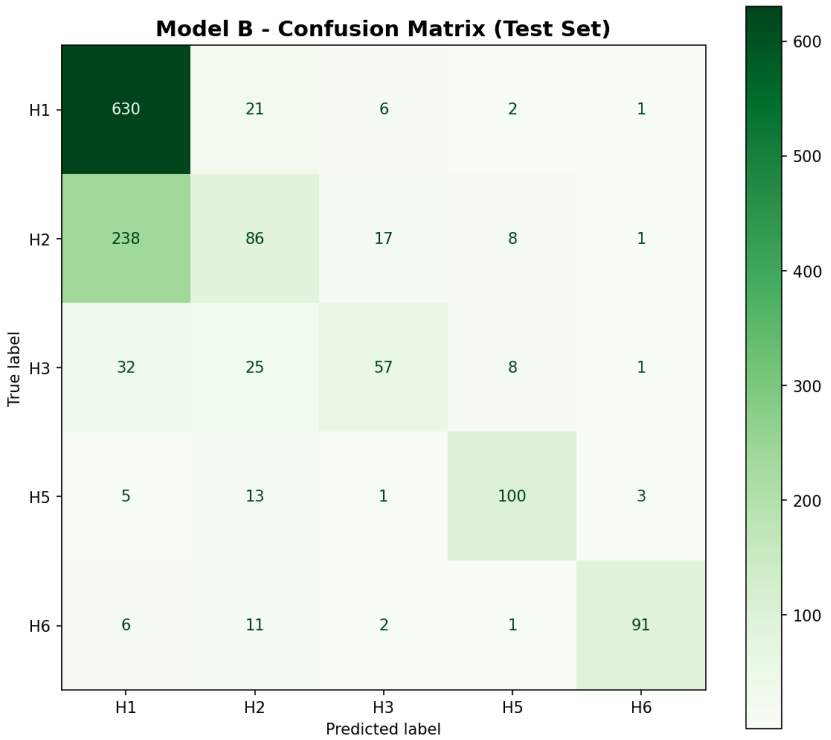


(b) Model B.

Figure 4: Training and validation loss and accuracy for the two custom models.



(a) Model A.



(b) Model B.

Figure 5: Corrected test-set confusion matrices; each totals 1,366 images.

Table 5: Custom-model performance on the leakage-free test split. Checkpoint sizes use binary MiB/KiB.

Model	Params	Size	Test loss	Test acc.	Macro P	Macro R	Macro F1	Total time	Time/ep	MACs	FLOPs
Model A	2,553,349	9.755 MiB	0.8360	67.64%	0.6820	0.6360	0.6254	346.43 s	17.32 s	62.325 M	124.651 M
Model B	37,957	179.8 KiB	0.6755	70.57%	0.7416	0.6606	0.6800	253.80 s	12.69 s	7.263 M	14.526 M

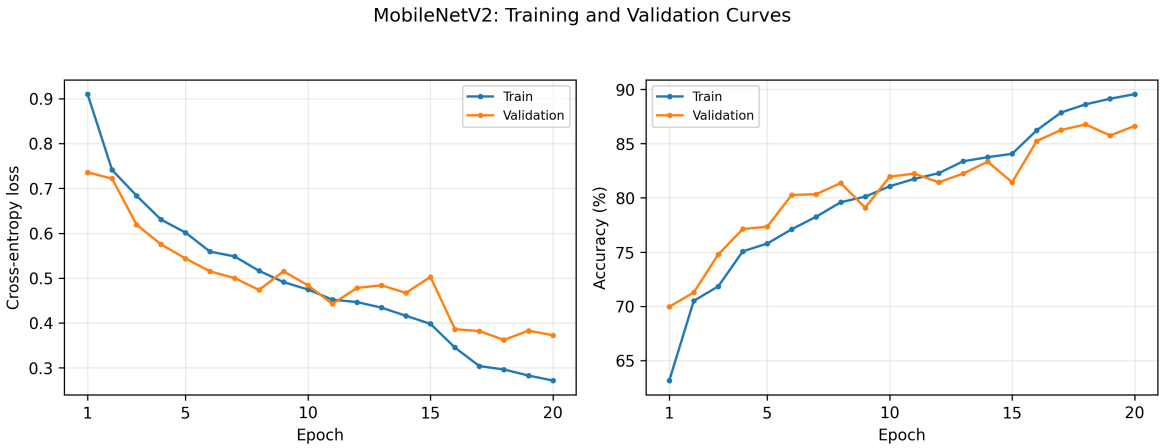
5.1 Model A versus Model B trade-off

Despite its much smaller size, Model B improved test accuracy by 2.93 percentage points and achieved a higher macro F1 (0.6800 versus 0.6254). It requires 98.51% fewer parameters equivalent to 67.27× fewer and 88.35% fewer calculated MACs. The checkpoint is also substantially smaller: 179.8 KiB rather than 9.755 MiB. In our training environment, its average epoch was shorter as well.

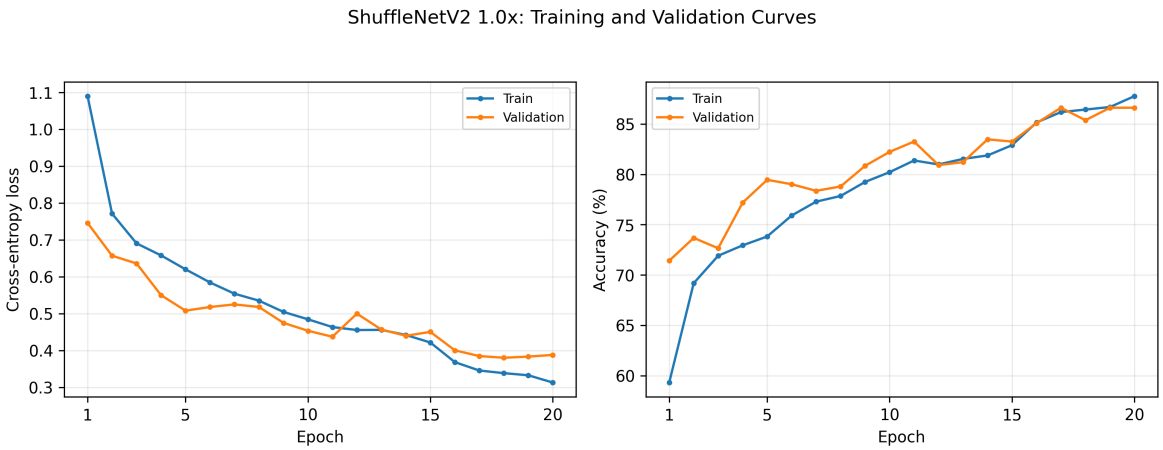
Model A remains a useful reference because its standard convolutions and large fully connected classifier are familiar and offer substantial representational capacity. However, that extra capacity did not translate into better accuracy on the clean test set. Model B's depthwise-separable layers and global pooling offer a more attractive balance for resource-constrained exploration. This finding should not be confused with a deployment guarantee: checkpoint size, parameter count, MACs, activation RAM, and inference latency describe different aspects of cost. We did not measure peak device RAM or inference latency.

6 Task 5: Lightweight Pre-trained Models

The custom models establish what can be achieved when training from scratch. We next considered whether architectures with ImageNet pre-training could offer better accuracy for a moderate increase in resources. MobileNetV2 was initialized with `torchvision.models.MobileNet_V2_Weights.DEFAULT`, and ShuffleNetV2 1.0x with `ShuffleNet_V2_X1_0_Weights.DEFAULT`. We replaced each original ImageNet classifier with a five-class Linear layer and fine-tuned all parameters using the same corrected split for 20 epochs. Both models used Adam with learning rate 5×10^{-4}, weight decay 10^{-4}, and `ReduceLROnPlateau` based on validation loss. For this task, we evaluated the checkpoint with the lowest validation loss.

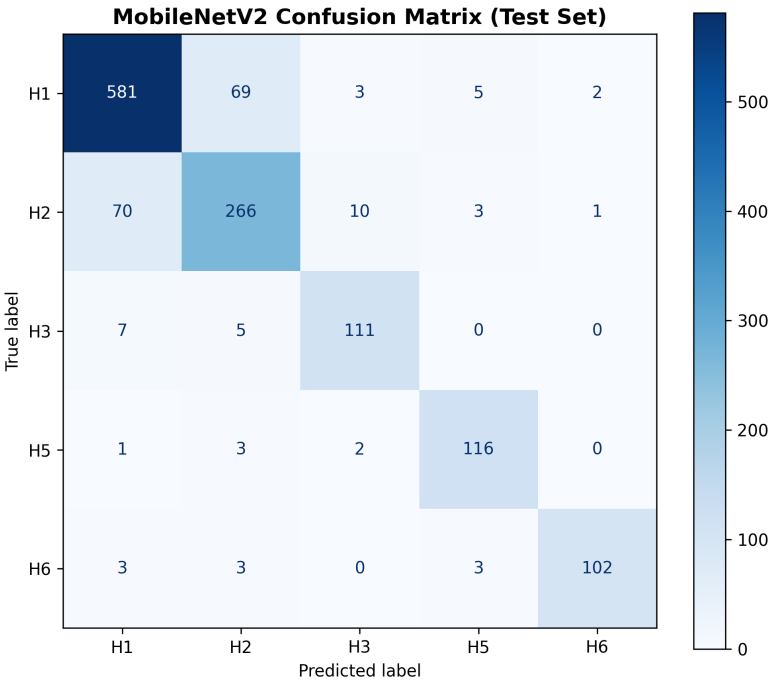


(a) MobileNetV2.

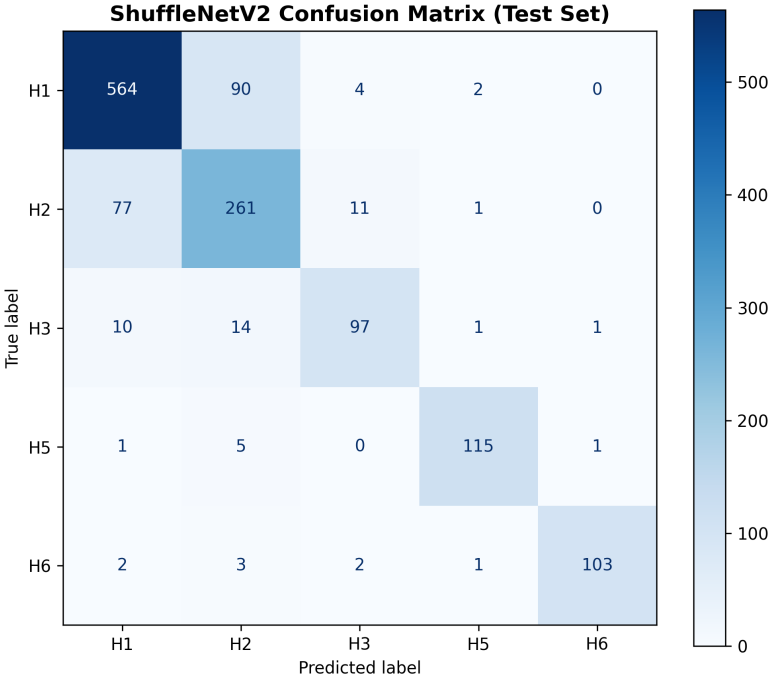


(b) ShuffleNetV2 1.0x.

Figure 6: Fine-tuning and validation curves for the two pre-trained models.



(a) MobileNetV2.



(b) ShuffleNetV2 1.0x.

Figure 7: Corrected test-set confusion matrices for the fine-tuned models.

Table 6: Fine-tuned model results on the leakage-free test split.

Model	Params	Size	Loss	Accuracy	Macro P	Macro R	Macro F1	Total time	Time/ep	MACs/FLOPs
MobileNetV2	2,230,277	8.740 MiB	0.3581	86.09%	0.8824	0.8825	0.8822	753.41 s	37.66 s	24.455/48.910 M
ShuffleNetV2 1.0x	1,258,729	4.966 MiB	0.3968	83.46%	0.8705	0.8519	0.8606	482.69 s	24.13 s	11.751/23.502 M

The fine-tuning results show a clear accuracy advantage over the custom models. MobileNetV2 achieves the highest accuracy and macro F1, while ShuffleNetV2 provides a smaller model with fewer MACs and still performs substantially better than Model B. Pre-training is likely helpful because the networks begin with reusable visual features; however, model capacity and architecture differ too, so this experiment cannot isolate the effect of pre-training alone.

7 Task 6: Final Comparison and Trade-off Discussion

7.1 Accuracy, storage, and computation

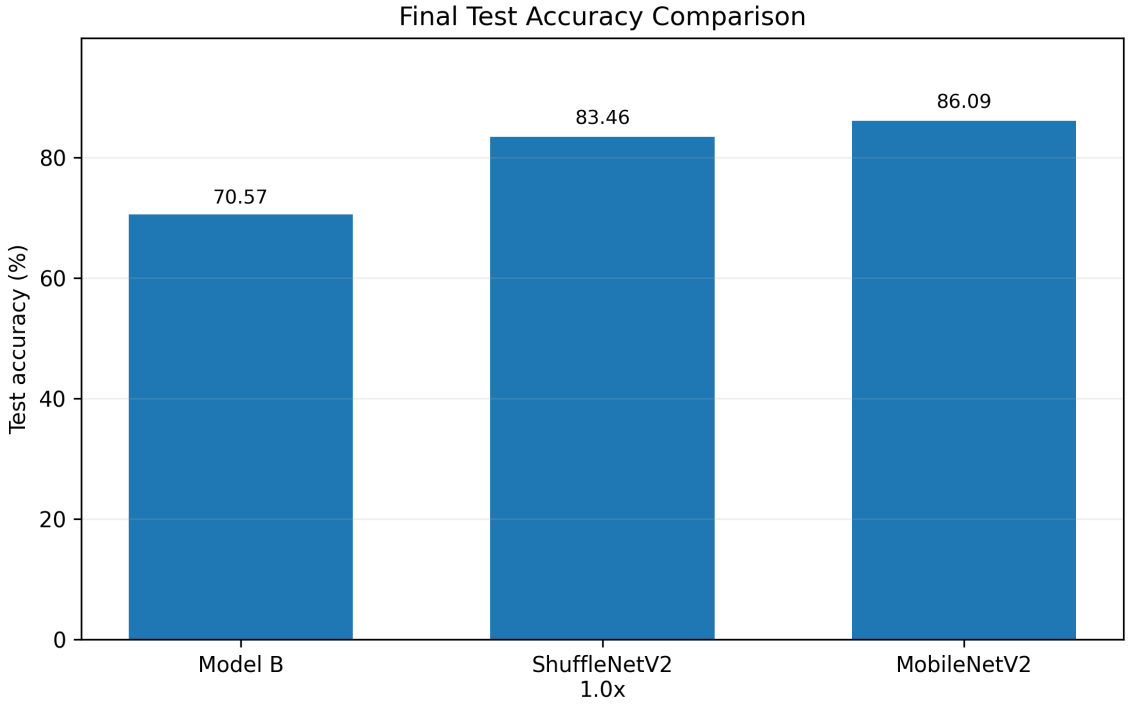
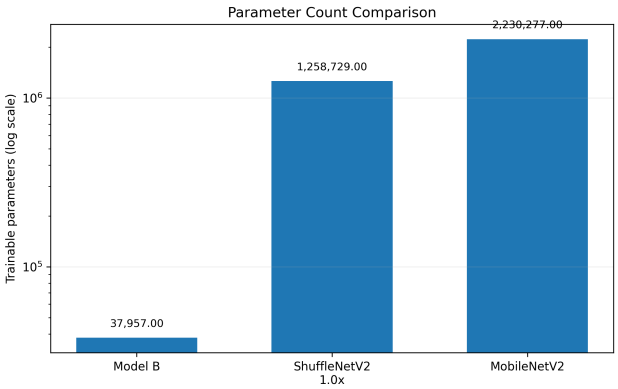
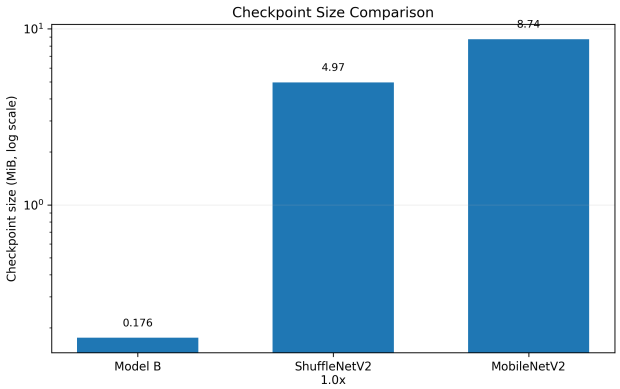


Figure 8: Corrected test accuracy for Model B and the two fine-tuned pre-trained models.

Looking at the held-out test set, MobileNetV2 exceeds Model B's accuracy by 15.52 percentage points, while ShuffleNetV2 gains 12.88 points. The improvement is not limited to the overall accuracy: for the H2 class, recall rises from 0.2457 in Model B to 0.7457 in ShuffleNetV2 and 0.7600 in MobileNetV2. This class-level comparison is especially useful given the unequal class frequencies.



(a) Trainable parameters.



(b) Checkpoint storage in MiB.

Figure 9: Parameter and actual checkpoint-size comparisons. Log scales expose the large ratios.

The accuracy gains come with greater storage needs. Compared with Model B, MobileNetV2 has 58.76× as many parameters and a 49.79× larger checkpoint. ShuffleNetV2 has 33.16× as many parameters and a 28.29× larger checkpoint. These file-size comparisons do not determine peak runtime RAM, which also depends on activation buffers, operators, the framework, and memory allocation.

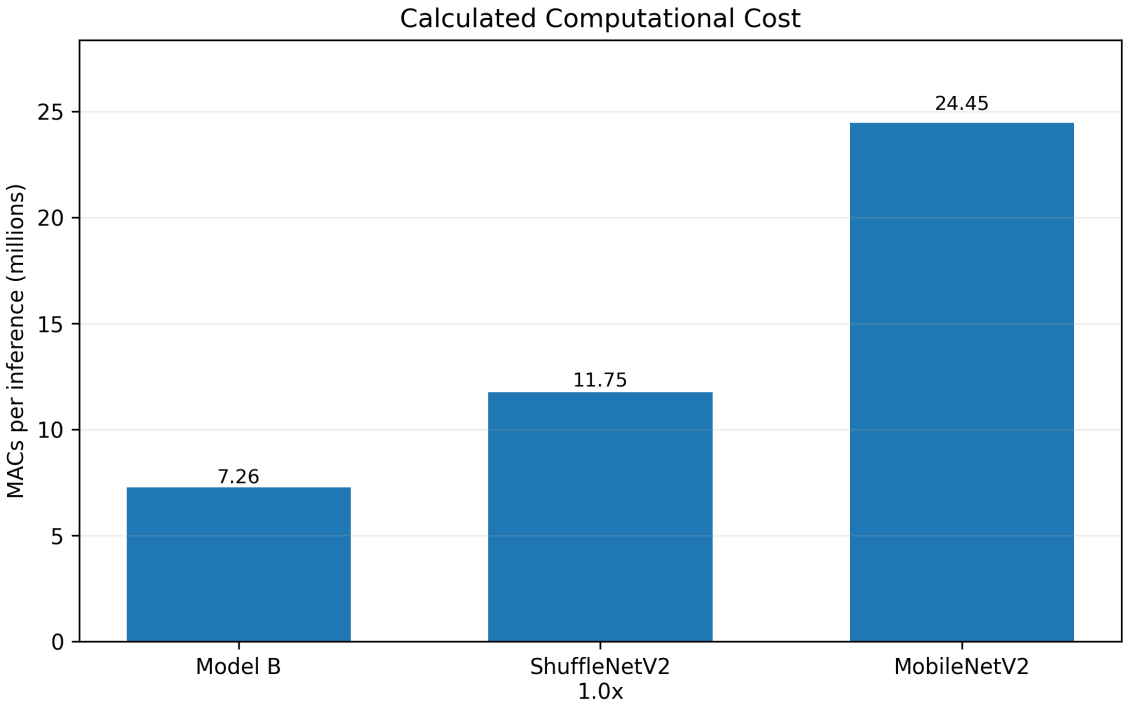
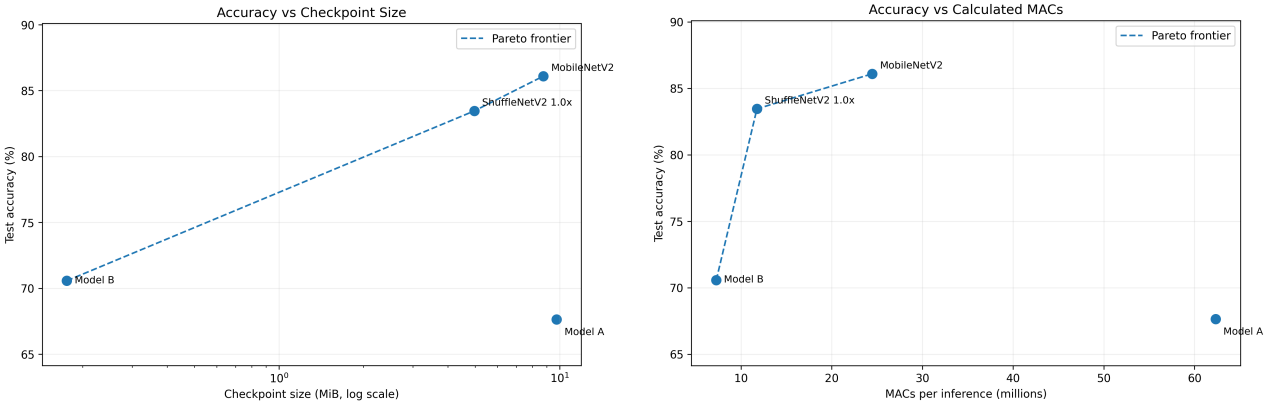


Figure 10: Calculated Conv2d/Linear MACs per 64 × 64 image.

Computationally, MobileNetV2 requires 3.37× Model B's MAC count, and ShuffleNetV2 requires 1.62×. Model B thus offers the lowest calculated arithmetic cost among these three models. The pre-trained alternatives spend more computation in exchange for better recognition, drawing on different efficient network blocks, greater capacity, and features learned from ImageNet. MAC ratios are useful for comparing workloads, but they are not direct predictions of device latency.

7.2 Accuracy-efficiency frontier

A single best model cannot be chosen from accuracy alone. To make the trade-off easier to see, we plot test accuracy against checkpoint storage and calculated MACs. Model A is dominated by Model B because the latter is both smaller and more accurate on this test set. The other three models occupy different points on the accuracy-efficiency frontier: Model B minimizes resources, ShuffleNetV2 offers a middle ground, and MobileNetV2 provides the highest measured accuracy.



(a) Accuracy versus checkpoint size.

(b) Accuracy versus MACs.

Figure 11: Corrected accuracy-efficiency trade-offs. Model A is dominated by Model B; the remaining three models provide distinct resource/accuracy operating points.

7.3 Engineering recommendations

- **Strictest parameter, storage, or arithmetic budget: Model B.** It is trained from scratch, contains only 37,957 parameters, and has the smallest checkpoint and MAC count. Quantization, peak-RAM profiling, operator support, and on-device latency must still be measured.
- **Balanced accuracy and resource cost: ShuffleNetV2 1.0x.** It reaches 83.46% accuracy with substantially fewer parameters, checkpoint storage, and MACs than MobileNetV2.
- **Accuracy-first choice: MobileNetV2.** It reaches 86.09% accuracy and macro F1 0.8822, the strongest measured classification result, with the highest compute cost among the three final candidates.

8 Conclusion

The experiments demonstrate that a carefully designed lightweight CNN can outperform a much larger conventional baseline. On the leakage-free split, Model B gives better classification results than Model A with 98.51% fewer parameters and 88.35% fewer MACs. Fine-tuned MobileNetV2 and ShuffleNetV2 achieve considerably higher accuracy, but this comes with increased resource requirements and reflects differences in architecture, capacity, and ImageNet pre-training.

For an edge application, the preferred model therefore depends on the actual accuracy target and limits on storage, RAM, latency, and energy. Our measurements cover checkpoint storage and training time, and our calculations estimate MACs; peak microcontroller RAM consumption and on-device

inference latency remain to be tested before deployment.

References

[1] UCI Machine Learning Repository, *DeFungi*, Dataset 773. `https://archive.ics.uci.edu/dataset/773/defungi`.

[2] M. Sandler et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *CVPR*, 2018.

[3] N. Ma et al., "ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design," *ECCV*, 2018.

[4] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *ICLR*, 2015.

[5] PyTorch and Torchvision model implementations, `https://pytorch.org/vision/stable/models.html`.